

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 3

Experiments

COURSE NAME: DEEP LEARNING

COURSE CODE: MLA0408

FACULTY : Dr. D.SUDHAGAR

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 50

Code of Conduct:

I **NALIPI REDDY ROHITH/192425115** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: N.Rohith

QUESTIONS:10

Total Marks:50

1. Learning Algorithms

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
X = np.array([1,2,3,4,5], dtype=float)
```

```
y = np.array([2,4,6,8,10], dtype=float)
```

```
m = 0
```

```
b = 0
```

```
lr = 0.01
```

```
epochs = 500
```

```
loss = []
```

```
n = len(X)
```

```
for i in range(epochs):
```

```
    y_pred = m*X + b
```

```
    error = y_pred - y
```

```
    dm = (2/n) * np.sum(error * X)
```

```
    db = (2/n) * np.sum(error)
```

```
    m -= lr * dm
```

```
    b -= lr * db
```

```
    loss.append(np.mean(error**2))
```

```
print("Slope:", m)
```

```
print("Intercept:", b)
```

```
plt.plot(loss)
```

```
plt.xlabel("Iterations")
```

```
plt.ylabel("Loss")
```

```
plt.title("Learning Curve")
```

```
plt.show()
```

2. Maximum Likelihood Estimation (MLE)

```
import numpy as np

np.random.seed(0)

true_mean = 5
true_std = 2

data = np.random.normal(true_mean, true_std, 1000)

mle_mean = np.mean(data)
mle_variance = np.mean((data - mle_mean)**2)

print("Actual Mean:", true_mean)
print("Estimated Mean:", mle_mean)

print("Actual Variance:", true_std**2)
print("Estimated Variance:", mle_variance)
```

3. Building Machine Learning Algorithms

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, confusion_matrix

data = load_iris()

X_train, X_test, y_train, y_test = train_test_split(
```

```
data.data,data.target,test_size=0.2,random_state=42)
```

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

```
model = LogisticRegression()
```

```
model.fit(X_train,y_train)
```

```
pred = model.predict(X_test)
```

```
print("Accuracy:",accuracy_score(y_test,pred))
```

```
print(confusion_matrix(y_test,pred))
```

4. Neural Networks

```
from sklearn.datasets import make_moons
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.neural_network import MLPClassifier
```

```
X,y = make_moons(n_samples=300,noise=0.2,random_state=0)
```

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2)
```

```
model=MLPClassifier(hidden_layer_sizes=(10,),max_iter=1000)
```

```
model.fit(X_train,y_train)
```

```
print("Accuracy:",model.score(X_test,y_test))
```

5. Artificial Neurons

```
import numpy as np
```

```
x = np.array([1,2,3])
```

```
w = np.array([0.2,0.4,0.6])
```

```
b = 0.5
```

```
z = np.dot(x,w)+b
```

```
sigmoid = 1/(1+np.exp(-z))
```

```
relu = max(0,z)
```

```
print("Neuron Output:",z)
```

```
print("Sigmoid:",sigmoid)
```

```
print("ReLU:",relu)
```

6. Perceptron and Multilayer Perceptron (MLP)

a)

```
from sklearn.datasets import make_classification
```

```
from sklearn.linear_model import Perceptron
```

```
X,y = make_classification(
```

```
    n_samples=100,
```

```
n_features=2,  
n_redundant=0,  
n_clusters_per_class=1,  
random_state=0)  
  
model = Perceptron()  
  
model.fit(X,y)  
  
print("Accuracy:",model.score(X,y))
```

b)

```
from sklearn.datasets import load_iris  
from sklearn.model_selection import train_test_split  
from sklearn.neural_network import MLPClassifier
```

```
iris = load_iris()
```

```
X_train,X_test,y_train,y_test = train_test_split(  
iris.data,iris.target,test_size=0.2)
```

```
mlp = MLPClassifier(hidden_layer_sizes=(20,),max_iter=1000)
```

```
mlp.fit(X_train,y_train)
```

```
print("MLP Accuracy:",mlp.score(X_test,y_test))
```

7. Forward Propagation and Backpropagation Algorithm

a)

```
import numpy as np
```

```
x = np.array([1,2])
```

```
w = np.array([[0.5,0.3],  
              [0.4,0.7]])
```

```
hidden = np.dot(x,w)
```

```
output = 1/(1+np.exp(-hidden))
```

```
print(output)
```

b)

```
import numpy as np
```

```
x = 1
```

```
target = 2
```

```
w = 0.5
```

```
lr = 0.1
```

```
output = w*x
```

```
error = output-target
```

```
grad = error*x
```

```
w = w-lr*grad
```

```
print("Updated Weight:",w)
```

8. Gradient Descent and Stochastic Gradient Descent (SGD)

a)

```
import numpy as np
```

```
x = 5
```

```
lr = 0.1
```

```
for i in range(20):
```

```
    grad = 2*x
```

```
    x = x-lr*grad
```

```
    print(i,x)
```

b)

```
from sklearn.linear_model import SGDRegressor
```

```
import numpy as np
```

```
X=np.array([[1],[2],[3],[4],[5]])
```

```
y=np.array([2,4,6,8,10])
```

```
model=SGDRegressor(max_iter=1000)
```

```
model.fit(X,y)
```

```
print(model.predict([[6]]))
```

9. Mini-Batch Gradient Descent

```
from sklearn.neural_network import MLPRegressor

import numpy as np

X=np.random.rand(100,2)

y=X[:,0]+X[:,1]

model=MLPRegressor(batch_size=16,max_iter=500)

model.fit(X,y)

print(model.score(X,y))
```

10. Curse of Dimensionality

```
import numpy as np

for d in [2,10,50,100,500]:

    X=np.random.rand(100,d)

    dist=[]

    for i in range(99):

        dist.append(np.linalg.norm(X[i]-X[i+1]))

    print("Dimension:",d)

    print("Average Distance:",np.mean(dist))
```


RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand